Class 6: R functions

Stephanie Solis (PID: A18572681)

Table of contents

Background

Akk functions in R have at least 3 things:

- A **name** that we use to call the function. _ One or more input **arguments**
- The **body** the lines of R code that do the work

Our first function

Let's write a silly wee functions called `add()` to add some numbers (the inpput arguments)

```
add <- function(x,y) {x+y}
```

Now we can use this function

```
add (x=100, y=1)
```

```
[1] 101
```

```
add (x=c(100,1,100),y=1)
```

```
[1] 101   2 101
```

Q. What if I give a multiple element vector to `x` and `y`

```
add (x=c(100,1),y=c(100,1))
```

```
[1] 200   2
```

Q. What if I give three inputs to the function?

```
#add(x=c(100,1),y=1, z=1)
```

Q. Whhat if I give only one input to the add function?

```
addnew <- function(x,y=1) {x+y}
```

```
addnew(x=100)
```

```
[1] 101
```

```
addnew(c(100,1), 100)
```

```
[1] 200 101
```

If we write our function with input arguments having no default value that the user will be required to set them when they use the function. We can give our input arguments "default" values by setting them equal to some sensible value -e.g. y=1 `addnew()` function

A second function

Let's try something more interesting: Make a sequence generating tool..

The `sample()` function can be a useful starting point

```
sample(1:10, size=4)
```

```
[1] 3 9 2 4
```

Q. Generate 9 random numbers taken form the input vector x=1:10

```
 sample(1:10, size=9)
```

```
[1]  6  7  2  9  5 10  1  8  3
```

Q. Generate 12 random numbers taken form the input vector x=1:10

```
sample(1:10, size=12, replace=TRUE)
```

```
 [1] 10 10  1  3 10  9  1  5  3  9  4  7
```

. Q. Write code for the `sample()` functions that generates nucleotides sequences of length 6?

```
sample(x=c("A","C","T","G"), size=6, replace=TRUE)
```

```
[1] "C" "G" "T" "A" "G" "C"
```

Q. Write a first function `generate_dna()` that returns a user specified length DNA sequence:

```
generate_dna<- function(len=6) {sample(x=c("A","C","T","G"), size=len, replace=TRUE)}
```

```
generate_dna(len=100)
```

```
  [1] "C" "A" "A" "T" "A" "T" "C" "G" "G" "G" "T" "T" "C" "T" "T" "C" "G" "A"
 [19] "C" "C" "T" "C" "T" "C" "A" "A" "C" "G" "A" "C" "T" "A" "G" "T" "G" "T"
 [37] "A" "G" "A" "T" "T" "G" "T" "A" "G" "G" "G" "G" "C" "A" "C" "G" "C" "C"
 [55] "A" "T" "A" "T" "A" "A" "C" "G" "G" "A" "C" "T" "T" "A" "A" "A" "A" "G"
 [73] "G" "T" "C" "T" "G" "A" "G" "G" "G" "G" "C" "A" "T" "A" "G" "T" "G" "T"
 [91] "C" "C" "G" "A" "G" "T" "T" "C" "A" "A"
```

Key-Points Every function in R looks fundamnetally the same in terms of its structure. Basically 3 things: name, input, and body

```
name <-function (input) {body}
```

Functions can ahve multiple inputs. These can be **required** arguments or **optional** arguments.

Q. Modify and improve our `generate_dna()` function to return it's genrated sequence in a more satndard format like " AGTAGTA" rather than the vector "A", "C", "G", "A"

```
generate_dna<- function(len=6, fasta=TRUE) {

  ans<- sample(x=c("A","C","T","G"),
               size=len, replace=TRUE)
  if(fasta) {cat( "Single-elecment vetcor ouput")
 ans<- paste(ans, collapse="")
  } else { cat ("Multi-elecment vector output")}


  return(ans)
  }

generate_dna(fasta=FALSE)
```

```
Multi-elecment vector output
```

```
[1] "C" "A" "T" "A" "C" "T"
```

The`paste()` function -it's job is to join up or stick together (a.k.a. paste) input strings together

```
 paste ("alice", "loves R", sep=" ")
```

```
[1] "alice loves R"
```

Flow control means where the R brain goes in your code

```
good_mood <- TRUE

if (good_mood) {cat("Great!")
  }else {
  cat("Bummer!") }
```

```
Great!
```

A protein generating function

> Q. Write a function, called `generate_protein()`, that generates a user specified length protein sequence.

```r
generate_protein <- function(len) {

  # The amino-acids to sample from
  aa <- c("A", "R", "N", "D", "C", "Q", "E", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T

  # Draw n=len amino acids to make our sequence
  ans <- sample(aa, size=len, replace=T)
  ans <- paste(ans, collapse = "")
  return(ans)
}
```

```r
generate_protein(len=44)
```

```
[1] "QKPYYSLADPYRFVFAMTVGHIYLDIRGCFRQEDFIHTETINRF"
```

Q. Use that function to generate random protein sequences between length 6 and 12

```r
generate_protein(6)
```

```
[1] "QSYGSS"
```

```r
generate_protein(7)
```

```
[1] "NGYRDIF"
```

```r
generate_protein(8)
```

```
[1] "EIKVTMSP"
```

```r
generate_protein(9)
```

```
[1] "DVCWAETKS"
```

```r
generate_protein(10)
```

```
[1] "HGMNAVVNMC"
```

```
generate_protein(11)
```

```
[1] "LKNKADAQTTM"
```

```
generate_protein(12)
```

```
[1] "AMGPGGGHITDT"
```

```
for(i in 6:12) {
  #FASTA ID line">id"
  cat(">",i, sep="", "\n")
  # Protein sequence line
  cat(generate_protein(i), "\n")
}
```

```
>6
PFGYND
>7
VCFNFSM
>8
DAEDQMHW
>9
ELTIIYREY
>10
DSYDKQKPKC
>11
EHGYVRKPNLT
>12
HLGGCNNTHQLT
```

Q. Are any of those sequnces unique i.e. not found anywhere in nature?

Sequence 9